

Projeto 3: Aprendizado por Reforço

Prazo: quinta-feira, 12-11-2025, 23:59 PT.

ÍNDICE

- IntroduçãoMDPsPergunta 1 (5 pontos): Iteração de ValorPergunta 2 (1 ponto): Análise de Travessia de PontesPergunta 3 (6 pontos): PolíticasPergunta 4 (1 ponto): Iteração de Valor Prioritário[Crédito Extra]Pergunta 5 (5 pontos): Q-AprendizagemPergunta 6 (2 pontos): Epsilon GreedyPergunta 7 (1 ponto): Travessia de Ponte RevisitadaPergunta 8 (1 ponto): Q-Aprendizagem e PacmanPergunta 9 (4 pontos): Aproximado Q-AprendizagemEnvio
-
-
-
-
-
-

Introdução

Neste projeto, você implementará iteração de valor e Q-learning. Você testará seus agentes primeiro no Gridworld (a partir da aula), depois aplicá-los a um controlador robótico simulado (Crawler) e, Pacman.As em projetos anteriores, este projeto inclui um autogradador para você avaliar suas soluções na sua máquina. Isso pode ser executado em todas as perguntas com o comando:

```
Python autograder.py
```

Pode ser executado para uma pergunta específica, como q2, por:

```
Python autograder.py -q Q2
```

Ele pode ser executado para um teste específico por comandos do formulário:

```
Python autograder.py -t test_cases/q2/1-bridge-grid
```

O código deste projeto contém os seguintes arquivos, disponíveis como arquivo postal.

Arquivos que você vai editar:

<code>valueIterationAgents.py</code>	Um agente de iteração de valor para resolver MDPs conhecidos.
<code>qlearningAgents.py</code>	Agentes Q-learning para Gridworld, Crawler e Pacman.
<code>analysis.py</code>	Um arquivo para colocar suas respostas às perguntas feitas no projeto.

Arquivos que você pode querer dar uma olhada:

<code>mdp.py</code>	Define métodos sobre MDPs gerais.
<code>learningAgents.py</code>	Define as classes base ValueEstimationAgent e QLearningAgent, que seus agentes irão expandir.
<code>util.py</code>	Utilidades, incluindo utilidades. Counter, que é particularmente útil para aprendizes Q.
<code>gridworld.py</code>	A implementação do Gridworld.
<code>featureExtractors.py</code>	Classes para extrair características em pares (estado, ação). Usado para o agente de aprendizagem Q-aproximado (em <code>qlearningAgents.py</code>).

Arquivos de suporte que você pode ignorar:

<code>environment.py</code>	Aula resumida para ambientes gerais de aprendizado por reforço. Usado por <code>gridworld.py</code> .
<code>graphicsGridworldDisplay.py</code>	Exibição gráfica do Gridworld.
<code>graphicsUtils.py</code>	Utilitários gráficos.
<code>textGridworldDisplay.py</code>	Plug-in para a interface de texto do Gridworld.

<code>crawler.py</code>	O código do crawler e o chicote de teste. Você vai rodar isso, mas não editar.
<code>graphicsCrawlerDisplay.py</code>	GUI para o robô rastejante.
<code>autograder.py</code>	Projeto autogradador
<code>testParser.py</code>	Analisa arquivos de teste e solução do autogradador
<code>testClasses.py</code>	Aulas gerais de avaliação automática
<code>test_cases/</code>	Diretório contendo os casos de teste para cada questão
<code>reinforcementTestClasses.py</code>	Classes específicas de teste de autoavaliação do Projeto 3

Arquivos para editar e enviar: Você preencherá partes de `valueIterationAgents.py`, `qlearningAgents.py` e `analysis.py` durante a tarefa. Depois de concluir a tarefa, você enviará esses arquivos para o Gradescope (por exemplo, você pode enviar todos os arquivos `.py` na pasta). Por favor, não altere os outros arquivos desta distribuição. **Avaliação:** Seu código será autoavaliado quanto à correção técnica. Por favor, não altere os nomes de nenhuma função ou classe fornecida dentro do código, ou você causará um caos no autograder. No entanto, a correção da sua implementação – não os julgamentos do autogradador – será o juiz final da sua pontuação. Se necessário, revisaremos e corrigiremos as tarefas individualmente para garantir que você receba o devido crédito pelo seu trabalho.

Desonestidade Acadêmica: Vamos verificar seu código com outras submissões da aula redundância forlogica. Se você copiar o código de outra pessoa e enviar com pequenas alterações, saberemos. Esses detectores de trapagens são bem difíceis de enganar, então, por favor, não tente. Confiamos que todos vocês enviarão apenas seu próprio trabalho; Por favor, não nos decepcione. Se você o fizer, buscaremos as consequências mais fortes que possamos. **Buscando Ajuda:** Você não está sozinho! Se você se sentir travado em algo, entre em contato com a equipe do curso para obter ajuda. Horário de atendimento, seção e fórum de discussão estão disponíveis para seu apoio; Por favor, use-os. Se você não puder comparecer ao nosso horário de atendimento, avise e agendaremos mais. Queremos que esses projetos sejam gratificantes e instrutivos, não frustrantes e desmoralizantes. Mas não sabemos quando ou como ajudar a menos que você pergunte. **Discussão:** Por favor, tome cuidado para não postar spoilers. Tópicos necessários para este projeto:

- MDPS I: Vídeo 8, Nota 4.1-
- 4.2MDPS E: Vídeo 9, Nota 4.3-
- 4.5RLI I: Vídeo 10, Nota 5.1-5.3

- RL II: Vídeo 11, Nota 5.4-5.5

MDPs

Para começar, execute o Gridworld no modo de controle manual, que usa as setas:

```
Python gridworld.py -m
```

Você verá o layout de duas saídas na aula. O ponto azul é o agente. Note que quando você faz pressup, o agente só se move para o norte 80% do tempo. Essa é a vida de um agente do Gridworld! Você pode controlar muitos aspectos da simulação. Uma lista completa de opções está disponível executando:

```
Python gridworld.py -H
```

O agente padrão se move aleatoriamente

```
python gridworld.py -g MazeGrid
```

Você deve ver o agente aleatório pulando pela grade até que ele apareça em uma saída. Não é o melhor momento para um agente de IA. Nota: O MDP do Gridworld é tal que você deve primeiro entrar em um estado pré-terminal (as caixas duplas mostradas na interface gráfica) e então realizar a ação especial de 'sair' antes do episódio realmente terminar (no verdadeiro estado terminal chamado `TERMINAL_STATE`, que não é mostrado na interface gráfica). Se você rodar um episódio manualmente, seu retorno total pode ser menor do que você esperava, devido à taxa de desconto (`-d tochange; 0,9` por padrão). Olhe para a saída do console que acompanha a saída gráfica (ou use `-t` para todo o texto). Você será informado sobre cada transição que o agente experimenta (para desligar isso, use `-q`). Como no Pacman, as posições são representadas por coordenadas cartesianas (x, y) e quaisquer arrays são indexados por `[x][y]`, sendo 'norte' a direção do aumento de y, etc. Por padrão, a maioria das transições receberá uma recompensa de zero, embora você possa alterar isso com a opção de recompensa viva (`-r`).

Pergunta 1 (5 pontos): Iteração de Valor

Lembre-se da equação de atualização do estado de iteração de valor:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

Escreva um agente de iteração de valor no `ValueIterationAgent`, que foi parcialmente especificado para você em `valueIterationAgents.py`. Seu agente de iteração de valor é um planejador offline, não um agente de aprendizado por reforço, e portanto a opção de treinamento relevante é o número de iterações de iteração de valor que ele deve rodar (opção `-i`) em sua fase inicial de planejamento. `ValueIterationAgent` faz uma `onconstruction` MDP e executa iteração de valor para o número especificado de iterações antes que o construtor retorne.

A iteração de valores calcula V_k e Q_k imitativas em k -passos dos valores ótimos, V_k, Q_k
`runValueIteration`, implemente os seguintes métodos para o `ValueIterationAgent`:

- `computeActionFromValues(state)` calcula a melhor ação de acordo com a função valor dada pelos valores-próprios.
- `computeQValueFromValues(estado, ação)` retorna o Q-valor do par (estado, ação) dado pela função valor dada por `self.values`.

Essas quantidades são todas exibidas na interface gráfica: valores são números em quadrados, Q-valores são números em quartos quadrados, e políticas são setas para a saída de cada quadrado. Importante: Use a versão "batch" da iteração de valor, onde cada vetor é calculado a partir de um vetor fixo (como na aula), não a versão "online" onde um único vetor de pesos é atualizado no local. Isso significa que, quando o valor de um estado é atualizado em iteração com base nos valores de seus estados sucessores, os valores do estado sucessor usados no cálculo de atualização de valor devem ser aqueles da iteração $k-1$ (mesmo que alguns dos estados sucessores já tenham sido atualizados na iteração k). A diferença é $V_k - V_{k-1}$ (Sutton & Barto, no Capítulo 4.1, na página 91).

Nota: Uma política sintetizada a partir de valores de profundidade k (que refletem o próximo k recompensas) na verdade vão

Refletir as próximas $k+1$ recompensas (ou seja, V_{k+1} evolui). Da mesma forma, os valores Q também refletirão uma recompensa a mais (Q_{k+1} os valores (ou seja, você devolve). Você deve devolver a apólice sintetizada. Q_{k+1}

Dica: Você pode opcionalmente usar o utilitário. Classe contadora em `util.py`, que é um dicionário com valor padrão zero. No entanto, tome cuidado com o `argMax`: o `argmax` real que você quer pode ser uma chave que não está no contador! Nota: Certifique-se de lidar com o caso quando um estado não tiver ações disponíveis em um MDP (pense no que isso significa para recompensas futuras). Para testar sua implementação, execute o autogradador:

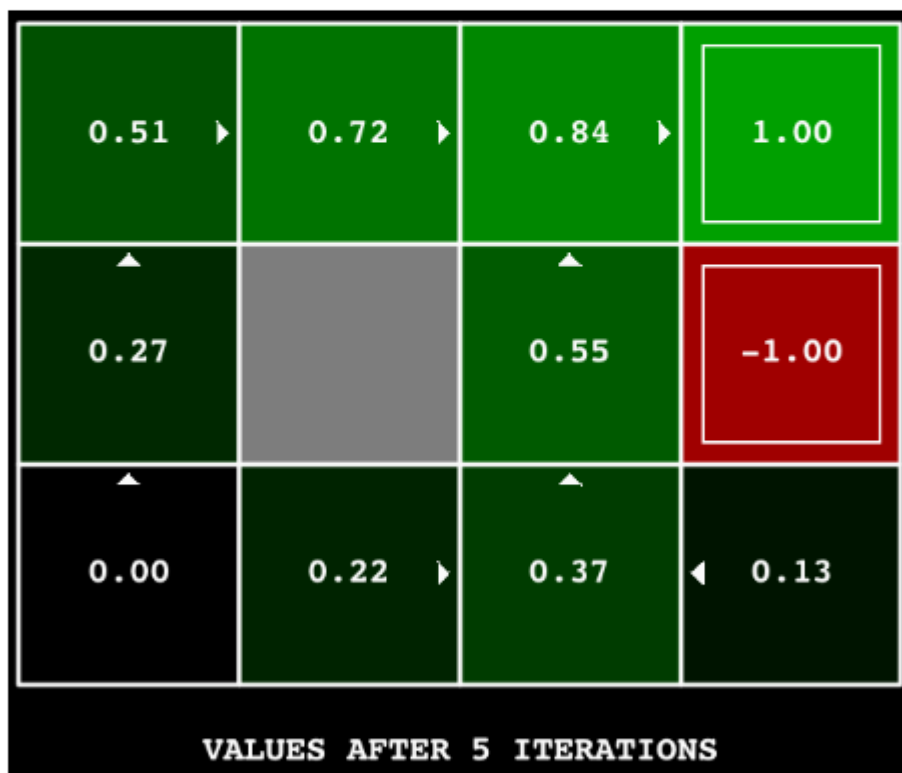
```
Python autograder.py -Q Q1
```

O comando a seguir carrega seu ValueIterationAgent, que irá calcular uma política e executá-la 10 vezes. Pressione uma tecla para alternar entre valores, valores Q e a simulação. Você deve perceber que o valor do estado inicial (V(início), que você pode ler na interface gráfica) e a média empírica resultante (impressa após as 10 rodadas de execução) são bem próximos.

```
Python gridworld.py -a valor -i 100 -k 10
```

Dica: No BookGrid padrão, rodar iteração de valor por 5 iterações deve te dar este resultado:

```
Python gridworld.py -a valor -i 5
```



Avaliação: Seu agente de iteração de valor será avaliado em uma nova grade. Vamos verificar seus valores, Q-valores e políticas após um número fixo de iterações e na convergência (por exemplo, após 100 iterações).

Pergunta 2 (1 ponto): Análise da Travessia de Ponte

BridgeGrid é um mapa-múndi em grade com um estado terminal de baixa recompensa e um estado terminal de alta recompensa, separados por uma estreita "ponte", de cada lado da qual há um abismo de alta recompensa negativa. O agente começa próximo ao estado de baixa recompensa. Com o desconto padrão de 0,9 e o ruído padrão de 0,2, a política ideal não é resolvida. Alterar apenas UM dos parâmetros de desconto e ruído para que a política ideal faça o agente tentar cruzar a ponte. Coloque sua resposta na pergunta2() de analysis.py . (Ruído refere-se a quantas vezes um agente acaba em um estado sucessor não intencional ao realizar uma ação.) O padrão corresponde a:

```
python gridworld.py -a valor -i 100 -g BridgeGrid --desconto 0,9 --ruído 0,2
```



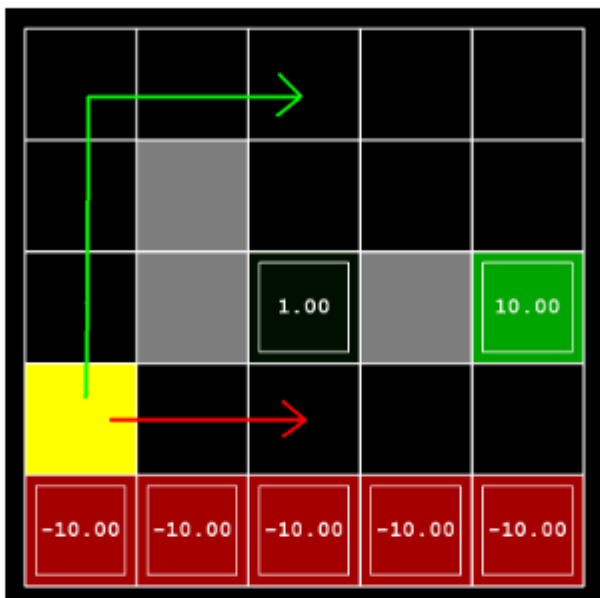
Gradação: Vamos verificar se você só alterou um dos parâmetros dados e que, com essa alteração, um agente de iteração de valor correto deve cruzar a ponte. Para verificar sua resposta, execute o autograder:

```
Python autograder.py -q Q2
```

Pergunta 3 (6 pontos): Políticas

Considere o layout do DiscountGrid, mostrado abaixo. Essa grade possui dois estados terminais com payoff positivo (na linha do meio), uma saída próxima com payoff +1 e uma saída distante com payoff +10. A linha inferior da grade consiste em estados terminais com pagamento negativo (mostrado em vermelho); Cada estado nessa região do "penhasco" tem um retorno de -10. O estado inicial é o quadrado amarelo. Distinguimos dois tipos de caminhos: (1) caminhos que "arriscam o penhasco" e passam próximos à fileira inferior da grade; Esses caminhos são mais curtos, mas correm o risco de gerar um grande retorno negativo, e são representados pelo

Seta vermelha na figura abaixo. (2) caminhos que "evitam o penhasco" e seguem ao longo da borda superior da grade. Esses caminhos são mais longos, mas têm menos chance de gerar grandes retornos negativos. Esses caminhos são representados pela seta verde na figura abaixo.



Nesta pergunta, você escolherá configurações dos parâmetros de desconto, ruído e recompensa em vida para este MDP para produzir apólices ótimas de vários tipos diferentes. Sua definição dos valores de parâmetros para cada parte deve ter a propriedade de que, se seu agente seguisse sua política ótima sem estar sujeito a nenhum ruído, ele exibiria o comportamento dado. Se um determinado comportamento não for alcançado para qualquer configuração dos parâmetros, afirme que a política é impossível retornando a cadeia 'NOT POSSIBLE'. Aqui estão os tipos de apólices ideais que você deve tentar produzir:

- 1 Prefira a saída próxima (+1), correndo o risco do penhasco (-10)
- 2 Prefiro a saída próxima (+1), mas evitando o penhasco (-10)
- 3 Prefira a saída distante (+10), correndo o risco do penhasco (-10)
- 4 Prefira a saída distante (+10), evitando o penhasco (-10)
- 5 Evite tanto as saídas quanto o penhasco (para que um episódio nunca deva terminar)

Para ver em que comportamento um conjunto de números acaba assumindo, execute o seguinte comando para ver uma interface gráfica:

```
python gridworld.py -g DiscountGrid -um valor --desconto [YOUR_DISCOUNT] --ruído [YOUR_NOISE] --livingRe
□□
```

Para verificar suas respostas, execute o autonivelador:

A `Question2A()` até a `Question2e()` devem devolver cada uma uma tupla de 3 itens de `(Desconto, Ruído, Recompensa Vital)` em `analysis.py`.

Nota: Você pode verificar suas políticas na interface gráfica. Por exemplo, usando uma resposta correta para 3(a), a seta em (0,1) deve apontar para o leste, a seta em (1,1) também deve apontar para o leste, e a seta em (2,1) deve apontar para o norte. Nota: Em algumas máquinas você pode não ver uma seta. Nesse caso, pressione um botão no teclado para mudar para a exibição de `qValor` e calcule mentalmente a política pegando o `arg max` dos `qValues` disponíveis para cada estado. Avaliação: Verificaremos se a apólice desejada é devolvida em cada caso.

Pergunta 4 (1 ponto): Iteração de Valor Geral Priorizado[Crédito Extra]

Esta pergunta é ponto extra. Planejamos priorizar os alunos que tiverem dúvidas sobre as partes obrigatórias deste projeto no Horário de Atendimento. Agora você implementará o `PriorSweepingValueIterationAgent`, que foi parcialmente especificado para você em `valueIterationAgents.py`. Priorizou tentativas abrangentes de focar atualizações dos valores estaduais de maneiras que provavelmente mudarão a política. Para este projeto, você implementará uma versão simplificada do algoritmo padrão de varrimento priorizado, que é descrito neste artigo. Adaptamos esse algoritmo para o nosso cenário. Primeiro, definimos os predecessores de um estado s como todos os estados que têm probabilidade diferente de zero de alcançar s ao realizar alguma ação a . Além disso, θ , que é passado como parâmetro, representará nossa tolerância ao erro ao decidir se atualizamos o valor de um estado. Aqui está o algoritmo que você deve seguir na sua implementação.

- Calcule predecessores de todos os estados. Inicialize uma fila de prioridade vazia. Para
 - cada estado não terminal s , faça: (nota: para fazer o autograder funcionar para esta
 - pergunta, você deve iterar sobre estados na ordem retornada por `self.mdp.getStates()`)
-
- Encontre o valor absoluto da diferença entre o valor atual de s em `self.valores` e o valor Q mais alto em todas as ações possíveis de s (isso representa qual deveria ser o valor); Chame esse número de diferença. NÃO atualize os valores-próprios nesta etapa.

Empurra s para a fila de prioridade com prioridade $-diff$ (note que isso é negativo).

Usamos $-diff$ porque a fila de prioridade é um mini heap, mas queremos priorizar estados de atualização que tenham erro maior. Para iterações em 0, 1, 2, ..., auto-

- iterações - 1, faça:

- Se a fila de prioridade estiver vazia, então termine. Tire um s
- estadual da fila de prioridade. Atualize o valor de s (se não for
- um estado terminal) em `self.valores`. Para cada predecessor p
- de s , faça:

- Encontre o valor absoluto da diferença entre o valor atual de p em `self.valores` e o maior valor q em todas as ações possíveis de p (isso representa qual deveria ser o valor); Chame esse número de diferença. NÃO atualize `self.values[p]` nesta
- etapa. Se $diff > \theta$, empurra p para a fila de prioridade com $-diff$ (note que isso é negativo), desde que ele não exista já na fila de prioridade com prioridade igual ou menor. Como antes, usamos um negativo porque a fila de prioridade é um mini heap, mas queremos priorizar a atualização de estados com erro maior.

Algumas observações importantes sobre implementação:

• Ao calcular predecessores de um estado, certifique-se de armazená-los em um conjunto, não em uma lista, para evitar duplicados. Por favor, use `util.PriorityQueue` na sua implementação.

• O método de atualização nesta classe provavelmente será útil; Veja a documentação dele.

Para testar sua implementação, execute o autograder. Deve levar cerca de 1 segundo para rodar. Se demorar muito mais, você pode ter problemas mais adiante no projeto, então torne sua implementação mais eficiente agora.

```
Python autograder.py -q Q4
```

Você pode executar o `PrioritizedSweepingValueIterationAgent` no Gridworld usando o seguinte comando.

```
Python gridworld.py -a priosweepvalue -i 1000
```

Avaliação: Seu agente de iteração de valor varrido priorizado será avaliado em uma nova grade. Vamos verificar seus valores, Q -valores e políticas após um número fixo de iterações e na convergência (por exemplo, após 1000 iterações).

Pergunta 5 (5 pontos): Q-Learning

Note que seu agente de iteração de valor na verdade não aprende com a experiência. Em vez disso, reflete sobre seu modelo MDP para chegar a uma política completa antes mesmo de interagir com um ambiente real. Quando ele interage com o ambiente, ele simplesmente segue a política pré-computada (por exemplo, torna-se um agente reflexo). Essa distinção pode ser sutil em um ambiente simulado como o aGridworld, mas é muito importante no mundo real, onde o MDP real não está disponível. Agora você vai escrever um agente Q-learning, que faz muito pouco sobre construção, mas aprende por tentativa e erro a partir das interações com o ambiente através do método de atualização (estado, ação, nextEstado, recompensa). Um stub de um Q-learner é especificado em QLearningAgent em

```
qlearningAgents.py
```

, e você pode selecioná-lo com a opção `'-a q'`. Para esta questão, você deve implementar os métodos `update`, `computeValueFromQValues`, `getQValue`, e `computeActionFromQValues`.

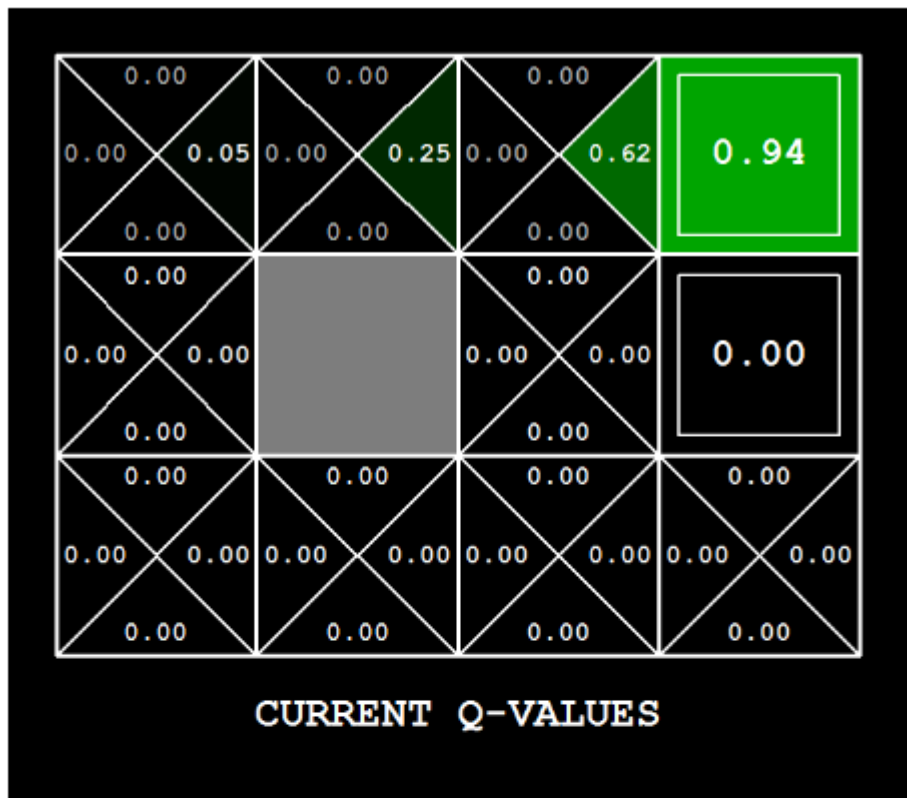
Nota: Para `computeActionFromQValues`, você deve romper os empates aleatoriamente para melhor comportamento. O

A função `random.choice()` vai ajudar. Em um estado específico, ações que seu agente nunca viu antes ainda têm um valor Q, especificamente um valor Q zero, e se todas as ações que seu agente já viu antes tiverem valor Q negativo, uma ação não vista pode ser ótima.

Importante: Certifique-se de que, nas suas funções `computeValueFromQValues` e `computeActionFromQValues`, você só acesse os valores Q chamando `getQValue`. Essa abstração será útil para a pergunta 10 quando você sobrescreve `getQValue` para usar características dos pares estado-ação em vez de pares estado-ação diretamente. Com a atualização Q-learning implementada, você pode assistir seu Q-learner aprender sob controle manual, usando o teclado:

```
Python gridworld.py -A Q -K 5 -M
```

Lembre-se que `-k` controla o número de episódios que seu agente pode aprender. Observe como o agente aprende sobre o estado em que estava, não sobre o que ele muda, e "deixa o aprendizado para trás." Dica: para ajudar na depuração, você pode desligar o ruído usando o parâmetro `--noise 0.0` (embora isso obviamente torne o aprendizado de Q menos interessante). Se você direcionar manualmente Pacman para o norte e depois para leste ao longo do caminho ótimo durante quatro episódios, deve ver os seguintes valores Q:



Correção: Vamos rodar seu agente Q-learning e verificar se ele aprende os mesmos Q-valores e políticas que nossa implementação de referência quando cada um for apresentado ao mesmo conjunto de exemplos. Para avaliar sua implementação, execute o autograder:

```
Python autograder.py -q Q5
```

Pergunta 6 (2 pontos): Epsilon Greedy

Complete seu agente de Q-learning implementando a seleção de ações gananciosa em epsilon em `getAction`, ou seja, ele escolhe ações aleatórias uma fração de ϵ das vezes, e segue seus melhores valores atuais de Q. Note que escolher uma ação aleatória pode resultar na escolha da melhor ação – ou seja, você não deve escolher uma ação aleatória subótima, mas sim qualquer ação legal aleatória. Você pode escolher um elemento de uma lista de forma uniforme e aleatória chamando a função `random.choice`. Você pode simular uma variável binária com probabilidade p de sucesso usando `util.flipCoin(p)`, que retorna Verdadeiro com probabilidade p e Falso com probabilidade $1-p$.

Após implementar o método `getAction`, observe o seguinte comportamento do agente em `GridWorld` (com $\epsilon = 0,3$).

```
Python gridworld.py -A Q -K 100
```

Seus Q-valores finais devem se assemelhar aos do seu agente de iteração de valor, especialmente ao longo de caminhos muito transitados. No entanto, seus retornos médios serão menores do que os valores Q previstos devido às ações aleatórias e à fase inicial de aprendizado. Você também pode observar as seguintes simulações para diferentes valores de ϵ . Esse comportamento do agente corresponde ao que você espera?

```
Python gridworld.py -A Q -K 100 --Ruído 0.0 -E 0.1
```

```
Python gridworld.py -A Q -K 100 --Ruído 0.0 -E 0.9
```

Para testar sua implementação, execute o autogradador:

```
Python autograder.py -Q Q6
```

Sem código adicional, agora você deve conseguir rodar um robô crawler Q-learning:

```
Python crawler.py
```

Se isso não funcionar, provavelmente você escreveu um código muito específico para o problema do `GridWorld` e deveria torná-lo mais geral para todos os MDPs. Isso vai invocar o robô rastejante da aula usando seu Q-learner. Brinque com os vários parâmetros de aprendizagem para ver como eles afetam as políticas e ações do agente. Note que o atraso de passo é um parâmetro da simulação, enquanto a taxa de aprendizado e a ϵ são parâmetros do seu algoritmo de aprendizado, e o fator de desconto é uma propriedade do ambiente.

Pergunta 7 (1 ponto): Travessia da Ponte Revisitada

Primeiro, treine um Q-learner completamente aleatório com a taxa padrão de aprendizado no `BridgeGrid` sem ruído por 50 episódios e observe se ele encontra a política ideal.

```
python gridworld.py -a q -k 50 -n 0 -g BridgeGrid -e 1
```

Agora tente o mesmo experimento com um ϵ de 0. Existe uma ϵ e uma taxa de aprendizado para as quais é altamente provável (maior que 99%) que a política ótima seja aprendida após 50 iterações? pergunta8() em analysis.py deve devolver OU uma tupla de 2 itens (ϵ , taxa de aprendizagem) OU a sequência 'NÃO POSSÍVEL' se não houver nenhuma. ϵ é controlada por -e , taxa de aprendizado por -l . Nota: Sua resposta não deve depender exatamente do mecanismo de desempate usado para escolher as ações. Isso significa que sua resposta deve estar correta mesmo se, por exemplo, girássemos toda a grade do mundo da ponte em 90 graus. Para avaliar sua resposta, execute o autograder:

```
Python autograder.py -q Q7
```

Pergunta 8 (1 ponto): Q-Learning e Pacman

Hora de jogar um pouco de Pacman! Pacman jogará jogos em duas fases. Na primeira fase, o treinamento, o Pacman começará a aprender sobre os valores das posições e ações. Como leva muito tempo para aprender valores Q precisos mesmo para grids minúsculas, os jogos de treinamento do Pacman rodam no modo silencioso por padrão, sem display de interface gráfica (ou console). Quando o treinamento do Pacman estiver completo, ele entrará no modo de teste. Durante os testes, self.epsilon e self.alpha do Pacman serão definidos para 0.0, parando efetivamente o Q-learning e desabilitando a exploração, para permitir que o Pacman explore sua learnedpolicy. Jogos de teste são exibidos na interface gráfica por padrão. Sem nenhuma alteração no código, você deve conseguir rodar o Q-learning Pacman para grades muito pequenas da seguinte forma:

```
python pacman.py -p PacmanQAgent -x 2000 -n 2010 -l smallGrid
```

Note que o PacmanQAgent já está definido para você em termos do QLearningAgent que você já escreveu. O PacmanQAgent só é diferente porque possui parâmetros padrão de aprendizado que são mais eficazes para o problema do Pacman ($\epsilon=0,05$, $\alpha=0,2$, $\gamma=0,8$). Você receberá crédito total por essa pergunta se o comando acima funcionar sem exceções e seu agente vencer pelo menos 80% das vezes. O autoavaliador fará 100 jogos de teste após os jogos de treinamento de 2000. Dica: Se seu QLearningAgent trabalha para gridworld.py e crawler.py mas não parece estar aprendendo uma boa política para Pacman no smallGrid, pode ser porque seu `getAction` e/ou `computeActionFromQValues` não consideram, em alguns casos, adequadamente ações não vistas. Em particular, porque ações invisíveis têm, por definição, um valor Q zero, se todas as ações vistas tiverem valores Q negativos, uma ação não vista pode ser ótima. Cuidado com a `argMaxfunction` do util. Balcão!

Para avaliar sua resposta, corra:

Nota: Se você quiser experimentar com parâmetros de aprendizado, pode usar a opção -a , por exemplo -a $\epsilon = 0,1$, $\alpha = 0,3$ $\gamma = 0,7$. Esses valores serão então acessíveis como `self.epsilon` , `self.gamma` e `self.alpha` dentro do agente.

Nota: Embora um total de 2010 jogos sejam disputados, os primeiros 2000 jogos não serão exibidos devido à opção -x 2000, que designa os primeiros 2000 jogos para treinamento (sem resultado). Assim, você verá o Pacman jogar apenas os últimos 10 desses jogos. O número de jogos de treinamento também é passado para seu agente como a opção número de treinamento.

Nota: Se quiser assistir a 10 jogos de treinamento para ver o que está acontecendo, use o comando:

```
python pacman.py -p PacmanQAgent -n 10 -l smallGrid -a numTreinamento=10
```

Durante os treinos, você verá resultados a cada 100 jogos com estatísticas sobre como o Pacman está se saindo. Epsilon é positivo durante o treinamento, então Pacman joga mal mesmo depois de aprender uma boa política: isso porque ele ocasionalmente faz um movimento exploratório aleatório em um fantasma. Como referência, deve levar entre 1000 e 1400 jogos até que as recompensas do Pacman por um segmento de 100 episódios se tornem positivas, refletindo que ele começou a ganhar mais do que a perder. Ao final do treinamento, deve permanecer positivo e estar relativamente alto (entre 100 e 350). Certifique-se de entender o que está acontecendo aqui: o estado MDP é exatamente a configuração do board voltada para o Pacman, com as transições agora complexas descrevendo toda uma camada de mudança para esse estado. As configurações intermediárias do jogo em que Pacman se moveu mas o ghostshave não respondeu não são estados MDP, mas sim incluídas nas transições. Quando o Pacman terminar de treinar, ele deve vencer com muita confiança em jogos de teste (pelo menos 90% das vezes), já que agora ele está explorando sua política aprendida. No entanto, você vai perceber que treinar o mesmo agente no aparentemente simples MediumGrid não funciona bem. Em nossa implementação, as recompensas médias do treinamento do Pacman permanecem negativas durante todo o treinamento. Na época dos testes, ele joga mal, provavelmente perdendo todos os seus jogos de teste. O treinamento também vai levar muito tempo, apesar de sua ineficácia. Pacman não vence em layouts maiores porque cada configuração de tabuleiro é um estado separado com valores Q separados. Ele não tem como generalizar que encontrar um fantasma é ruim para todas as posições. Obviamente, essa abordagem não vai escalar.

Pergunta 9 (4 pontos): Aprendizagem Aproximada do Q

Implemente um agente aproximado de Q-learning que aprenda pesos para características de estados, onde muitos estados podem compartilhar as mesmas características. Escreva sua implementação na classe `ApproximateQAgent` em `qlearningAgents.py`, que é uma subclasse do `PacmanQAgent`. Nota: O Q-learning aproximado assume a existência de $f(s, a)$ ação de característica sobre pares de estado $[f(s, a), \dots, f(s, a), \dots, f(s, a)]$ em pares de características. Oferecemos funções funcionais para você em `featureExtractors.py`. Vetores de características são úteis. Objetos contadores (como um dicionário) contendo os pares não nulos de características e valores; todas as características omitidas têm valor zero. Então, em vez de um vetor onde o índice no vetor define qual característica é qual, temos as chaves no dicionário que definem a identidade da característica. A função Q aproximada assume a seguinte forma:

$$Q(s, a) = \sum_{i=1}^n w_i f_i(s, a)$$

onde cada peso w_i está associado a uma característica particular $f_i(s, a)$. No seu código, você deveria implementar o vetor de peso como um dicionário que mapeia características (que os extratores de características devolverão) aos valores de peso. Você atualizará seus vetores de peso de forma semelhante a como atualizou os valores Q:

$$w_i \leftarrow w_i + \alpha \cdot \text{diferença} \cdot f_i(s, a)$$

$$\text{diferença} = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$$

Note que o **diferença** termo é o mesmo que na aprendizagem normal do Q, e r é o experiente recompensa. Por padrão, o `ApproximateQAgent` usa o `IdentityExtractor`, que atribui uma única característica a cada

(estado, ação) par. Com esse extrator de características, seu agente de Q-learning aproximado deve funcionar da mesma forma que o `PacmanQAgent`. Você pode testar isso com o seguinte comando:

```
python pacman.py -p ApproximateQAgent -x 2000 -n 2010 -l smallGrid
```

Importante: `ApproximateQAgent` é uma subclasse de `QLearningAgent`, e portanto compartilha vários métodos como `getAction`. Certifique-se de que seus métodos no `QLearningAgent` chamem `getQValue` em vez de acessar os valores Q diretamente, para que, ao sobrescrever o `getQValue` no seu agente aproximado, os novos valores q aproximados sejam usados para calcular ações. Quando você tiver certeza de que seu aprendiz aproximado funciona corretamente com as características de identidade, execute seu agente aproximado de Q-learning com nosso extrator de características personalizado, que pode aprender a vencer com facilidade:


```
python pacman.py -p ApproximateQAgent -a extractor=SimpleExtractor -x 50 -n 60 -l mediumGrid
```

Mesmo layouts bem maiores não devem ser problema para seu ApproximateQAgent (aviso: isso pode levar alguns minutos para treinar):

```
python pacman.py -p ApproximateQAgent -a extractor=SimpleExtractor -x 50 -n 60 -l mediumClassic
```

Se você não tiver erros, seu agente aproximado de Q-learning deve vencer quase sempre com esses recursos simples, mesmo com apenas 50 jogos de treinamento. Avaliação: Vamos rodar seu agente aproximado de Q-learning e verificar se ele aprende os mesmos Q-valores e pesos de características da nossa implementação de referência quando cada um for apresentado com o mesmo conjunto de exemplos. Para avaliar sua implementação, execute o autograder:

```
Python autograder.py -q Q9
```

Parabéns! Você tem um agente Pacman aprendendo!

Submissão

Para enviar seu projeto, envie os arquivos Python que você editou. Por exemplo, o upload do useGradescope em todos os arquivos .py na pasta do projeto.
